Линейная алгебра в Julia

Лабораторная работа №4

Дагделен Зейнап Реджеповна

2026-10-22

Содержание I

1. Информация
2. Введение
3. Теоретическая часть
4. Прикладные задачи
5. Заключение

Раздел 1

Информация

Докладчик

- Дагделен Зейнап Реджеповна
- студентка НКНбд-01-23
- Российский университет дружбы народов им. П. Лумумбы
- 1132236052@rudn.ru



Раздел 2

Введение

Актуальность

- Линейная алгебра — фундамент машинного обучения, физики, экономики.
- Эффективность матричных вычислений критически важна.
- Язык **Julia** сочетает скорость C и удобство Python.
- Специализированные пакеты позволяют избежать ручной реализации алгоритмов.

Цели и задачи

- **Цель:** Изучить возможности пакетов Julia для линейной алгебры и оценить эффективность операций.
- **Задачи:**
  - Освоить базовые операции с матрицами и векторами.
  - Изучить факторизации (LU, QR) и спектральное разложение.
  - Провести бенчмаркинг вычислений.
  - Решить прикладные задачи (СЛАУ, модель Леонтьева).

Раздел 3

Теоретическая часть

Поэлементные операции

- Работают независимо с каждым элементом массива.
- Функции `sum`, `prod`, `mean` в пакете `Statistics`.
- Векторизация операций (`.` — броадкастинг) ускоряет вычисления.
- Позволяет избегать циклов `for`.

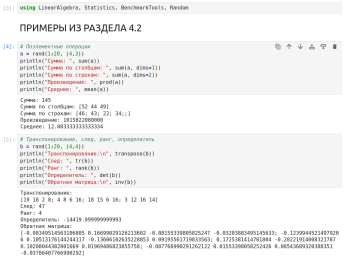


Рисунок 1: Поэлементные операции и операции над матрицами

Нормы и расстояния

- **Евклидова норма:** $\|\vec{X}\|_2 = \sqrt{\sum x_i^2}$
- **p-норма:** обобщение евклидовой нормы.
- **Расстояние** между векторами: $\|\vec{X} - \vec{Y}\|_2$.
- **Угол** между векторами через скалярное произведение.

```
[6]: # Нормы и расстояния
X = [2, 4, -5]
Y = [1, -1, 3]
println("Норма X (евклидова): ", norm(X))
println("p-норма (p=1): ", norm(X, 1))
println("Расстояние между X и Y: ", norm(X - Y))
println("Угол между векторами (рад): ", acos((X' * Y) / (norm(X) * norm(Y))))
```

```
Норма X (евклидова): 6.708203932499369
p-норма (p=1): 11.0
Расстояние между X и Y: 9.486832980505138
Угол между векторами (рад): 2.4404387889469252
```

```
[8]: #Умножение матриц, скалярное произведение,
     #единичная
A = rand(1:10, (2,3)); B = rand(1:10, (3,4))
println("A * B =\n", A * B)
I3 = Matrix{Int}(I, 3, 3) # Единичная матрица
println("Единичная 3x3:\n", I3)
println("Скалярное произведение dot: ", dot(X, Y))
println("Скалярное произведение X'Y: ", X'Y)
```

```
A * B =
[157 160 126 123; 154 136 105 90]
Единичная 3x3:
[1 0 0; 0 1 0; 0 0 1]
Скалярное произведение dot: -17
Скалярное произведение X'Y: -17
```

Рисунок 2: Нормы, расстояния и умножение матриц

Факторизации матриц

- **LU-разложение:** $A = LU$ (нижнетреугольная и верхнетреугольная).
- **QR-разложение:** $A = QR$ (ортогональная и верхнетреугольная).
- Применяются для решения СЛАУ и поиска собственных значений.
- В Julia реализованы в пакете `LinearAlgebra`.



Рисунок 3: Факторизации и спектральное разложение

Специальные структуры данных

- Julia поддерживает явное указание типа матрицы: `Symmetric`, `Diagonal`, `Tridiagonal`.
- Указание структуры позволяет алгоритмам **игнорировать проверки** и работать быстрее.
- «Зашумленная» матрица теряет свойства симметрии, что резко замедляет вычисления.

```
[10]: # BenchmarkTools (спец. структуры)
n = 1000
A = randn(n,n)
Asym = A + A'
Asym_noisy = copy(Asym)
Asym_noisy[1,2] += 5eps()
Asym_explicit = Symmetric(Asym_noisy)

# Сравнение скорости
println("Обычная симметричная:")
@btime eigvals(Asym);
println("Зашумленная:")
@btime eigvals(Asym_noisy);
println("Явно указанная Symmetric:")
@btime eigvals(Asym_explicit);
```

```
Обычная симметричная:
  128.718 ms (18 allocations: 7.98 MiB)
Зашумленная:
  724.310 ms (27 allocations: 7.93 MiB)
Явно указанная Symmetric:
  125.327 ms (18 allocations: 7.98 MiB)
```

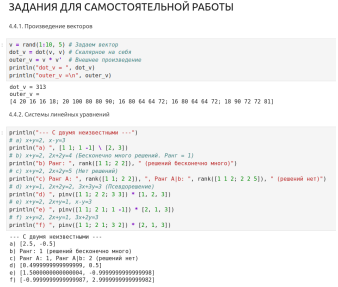
Рисунок 4: Оценка эффективности (BenchmarkTools)

Раздел 4

Прикладные задачи

СЛАУ и псевдообратная матрица

- Для квадратных систем используется оператор \.
- Для переопределенных систем — `pinv` (псевдообратная матрица).
- Проверка совместности через ранги матриц (`rank`).



ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

4.4.1. Произведение векторов

```
11]: v = rand(1:10, 5) # Задаем вектор
     dot_v = dot(v, v) # Скалярное на себя
     outer_v = v * v' # Внешнее произведение
     println("dot_v = ", dot_v)
     println("outer_v =\n", outer_v)
```

```
dot_v = 313
outer_v =
[4 20 16 16 18; 20 100 80 80 90; 16 80 64 64 72; 16 80 64 64 72; 18 90 72 72 81]
```

4.4.2. Системы линейных уравнений

```
12]: println("--- С двумя неизвестными ---")
     # a) x+y=2, x-y=3
     println("a) ", [1 1; 1 -1] \ [2, 3])
     # b) x+y=2, 2x+2y=4 (бесконечно много решений, Ранг = 1)
     println("b) Ранг: ", rank([1 1; 2 2]), " (решений бесконечно много)")
     # c) x+y=2, 2x+2y=5 (Нет решений)
     println("c) Ранг A: ", rank([1 1; 2 2]), ", Ранг A|b: ", rank([1 1 2; 2 2 5]), " (решений нет)")
     # d) x+y=1, 2x+2y=2, 3x+3y=3 (Псевдорешение)
     println("d) ", pinv([1 1; 2 2; 3 3]) * [1, 2, 3])
     # e) x+y=2, 2x+y=1, x-y=3
     println("e) ", pinv([1 1; 2 1; 1 -1]) * [2, 1, 3])
     # f) x+y=2, 2x+y=1, 3x+2y=3
     println("f) ", pinv([1 1; 2 1; 3 2]) * [2, 1, 3])
```

```
--- С двумя неизвестными ---
a) [2.5, -0.5]
b) Ранг: 1 (решений бесконечно много)
c) Ранг A: 1, Ранг A|b: 2 (решений нет)
d) [0.4999999999999999, 0.5]
e) [1.5000000000000004, -0.9999999999999998]
f) [-0.9999999999999987, 2.9999999999999982]
```

Рисунок 5: Системы линейных уравнений с двумя неизвестными

Диагонализация и спектр

- `eigen(A)` возвращает собственные значения и векторы.
- Диагональная матрица Λ строится через `Diagonal`.
- Спектральное разложение: $A = V\Lambda V^{-1}$.

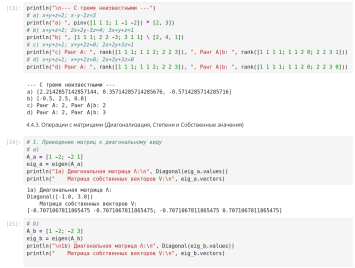


Рисунок 6: Приведение матриц к диагональному виду

Степени и корни матриц

- Возведение в степень (^) и извлечение корней (`sqrt`) работают для матриц.
- Корни могут давать **комплексные числа**, если матрица не имеет вещественного спектра.
- Это связано с комплексными собственными значениями.



Рисунок 7: Вычисления (степени и корни)

Модель Леонтьева

- Уравнение: $x - Ax = y$ (межотраслевой баланс).
- Матрица A **продуктивна**, если решение $x \geq 0$ при любом $y \geq 0$.
- **Критерий 2:** все элементы $(E - A)^{-1} \geq 0$.
- **Критерий 3 (спектральный):** $\max |\lambda| < 1$.

```
# 2 и 3. Проверка по критериям для матриц [1 2; 3 1] и их вариаций
println("--- 2 и 3. Проверка матриц из пунктов 2 и 3 ---")

# Задаем базовую матрицу для пунктов 2 и 3
M_base = [1.0 2.0; 3.0 1.0]

# а)
A_2a = M_base
check_criteria(A_2a, "2a/3a")

# б)
A_2b = (1/2) * M_base
check_criteria(A_2b, "2b/3b")

# в)
A_2c = (1/10) * M_base
check_criteria(A_2c, "2c/3c")

# 3г. Проверка матрицы 3x3
println("--- 3г. Матрица 3x3 ---")
A_3d = [
   0.1 0.2 0.3;
   0.0 0.1 0.2;
   0.0 0.1 0.3
]

check_criteria(A_3d, "3d")
```

```
--- 2 и 3. Проверка матриц из пунктов 2 и 3 ---

Матрица 2a/3a:
  Собственные значения: [-3.4494897427831779, 3.4494897427831783]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): false
  Критерий 3 (спектральный max|λ| < 1): false (max|λ| = 3.449)

Матрица 2b/3b:
  Собственные значения: [-0.7247448713915892, 1.7247448713915889]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): false
  Критерий 3 (спектральный max|λ| < 1): false (max|λ| = 1.725)

Матрица 2c/3c:
  Собственные значения: [-0.3449489742783178S, 0.3449489742783178З]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): true
  Критерий 3 (спектральный max|λ| < 1): true (max|λ| = 0.345)
```

Рисунок 8: Проверка продуктивности матриц по критериям

Раздел 5

Заключение

Главный вывод

Использование специализированных структур данных и встроенных функций Julia позволяет не только упростить код, но и **значительно ускорить** вычисления. Явное указание типа матрицы (например, `Symmetric`) дает выигрыш в скорости в разы по сравнению с обычными массивами.